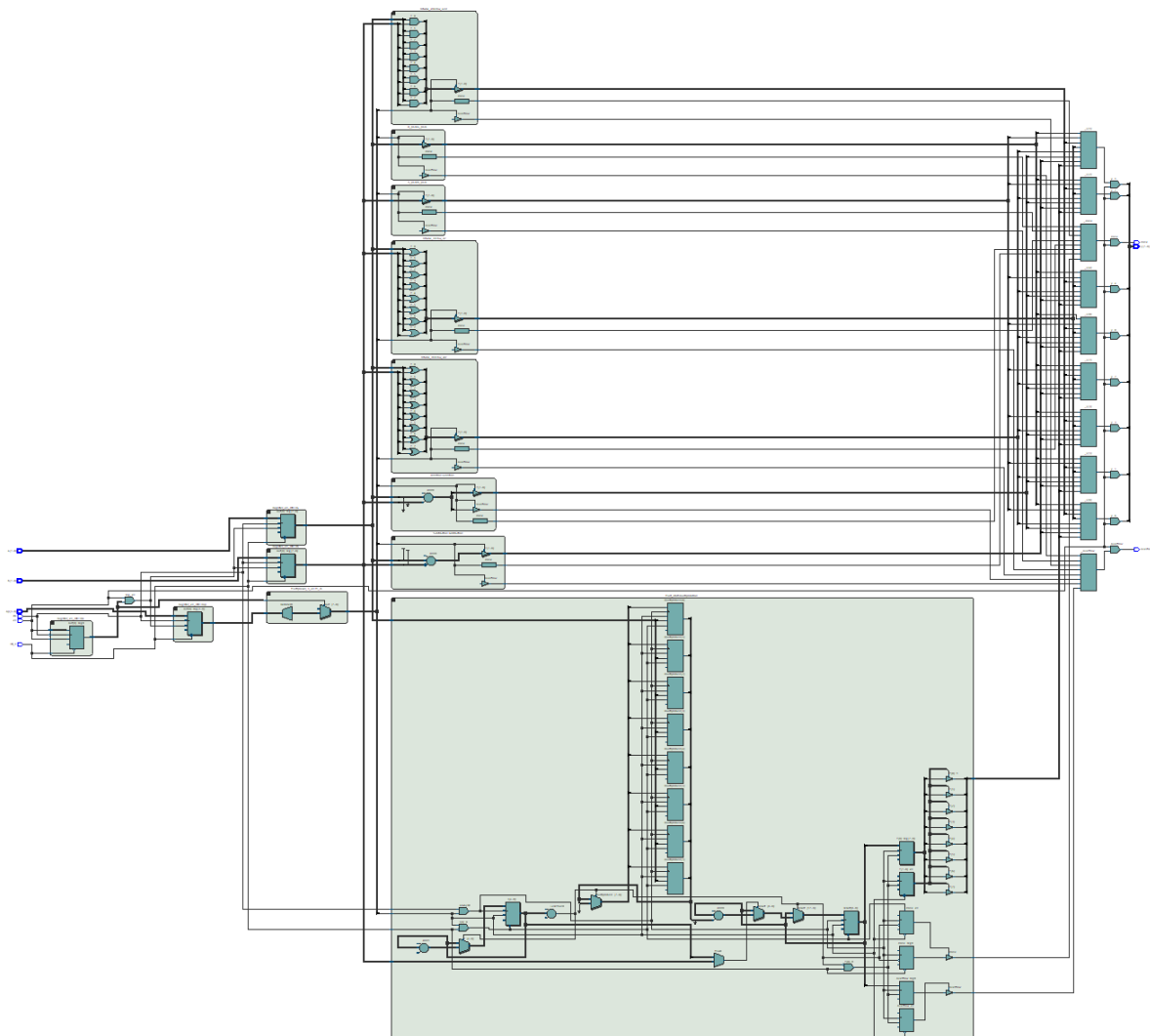


ECE 204 Design Project

8 Bit ALU

Nolan Kessler, Maxwell Fabian, John O'Connor

June 7, 2026



1 8 Bit ALU Functional Description

This 8 bit ALU accepts two 8 bit operands A and B, along with a 3 bit opcode (OP). The state of the ALU is controlled by an active high enable pin (EN), and can be reset using the active low reset pin (RST_N). As this ALU supports sequential operations which may take more than one clock cycle, the CLK input must be connected to a clock signal. The ALU provides one 8 bit result output (Y) an overflow flag (OVER) and a done flag (DONE). A table of the ALU's supported operations is given in table 1. A diagram of the inputs and outputs is shown in figure 1. A description of the function of each bus/pin is given in table 2.

Operation	Opcode	Clock Cycles	Overflow Condition	Description
A pass	0x0	1	N/A	Passes A to Y unmodified
B pass	0x1	1	N/A	Passes B to Y unmodified
Bitwise AND	0x2	1	N/A	Y is bitwise AND combination of operands A and B
Bitwise OR	0x3	1	N/A	Y is bitwise OR combination of operands A and B
Bitwise XOR	0x4	1	N/A	Y is bitwise XOR combination of operands A and B
Addition	0x5	1	$A + B > 255$	Y is the sum of operands A and B
Subtraction	0x6	1	$A + (\bar{B} + 1) > 255$	Y is the sum of A and the 2's complement inverse of B
Multiplication	0x7	9	$A * B > 255$	Y is the product of A and B

Table 1: Table of supported operations

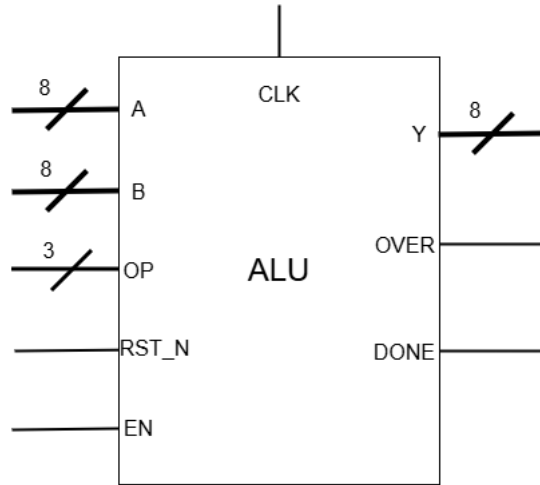


Figure 1: ALU Inputs and Outputs

Bus / Pin	Width (Bits)	Function
Input		
A	8	First 8-Bit operand - input to mathematical or logical operation.
B	8	Second 8-Bit operand - input to mathematical or logical operation.
OP	3	Opcode - selects which operation to perform.
RST_N	1	Active low asynchronous reset - puts the ALU into initial ready state.
EN	1	Enable - starts ALU when high if in ready state. Must be high for ALU to output data.
Output		
Y	8	8 Bit result - output of the ALU operation
OVER	1	Overflow flag - high value generally indicates result cannot be represented in bits of Y. Exact meaning can vary between operations.
DONE	1	Is low while operation is in progress or ALU is ready, high when operation is completed.

Table 2: ALU IO Functional Descriptions

1.1 ALU Operation

Before beginning an operation the ALU must be reset by setting RST_N low, after reset occurs the ALU is considered to be in the "ready" state. EN must be low for all rising edges of CLK while the ALU is in the ready state. Once the inputs on A, B, and OP are stable, EN can be set high to begin ALU operation. The rising edge of EN must occur when CLK is low. EN must remain high until after the next falling edge of CLK. At this point the ALU enters the "running" state. From this point on the value of EN can be changed without effecting the ALU (or the result when the ALU is done). The ALU will always output all zeros when EN is low. When DONE goes high the ALU is in the "done" state. The operation is finished, and results can be read from Y, and OVER. A timing diagram of a normal ALU operation is shown in figure 2.

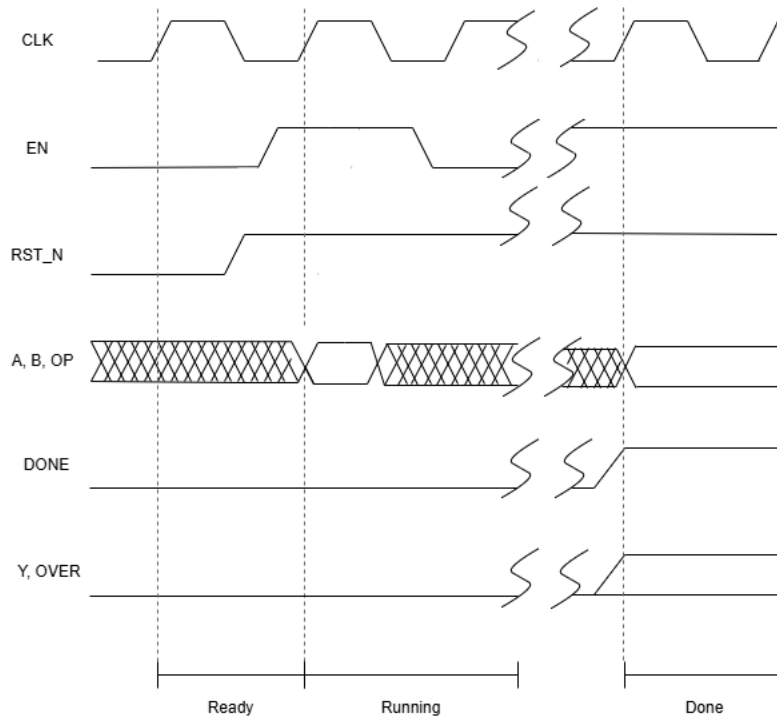


Figure 2: ALU Operation Timing Diagram

2 Design and Implementation

This ALU is designed to utilize modular operations, to allow the design to allow for easy testing during the design process, and potential rapid addition of new functions in the future. These modular operations are then linked together with integrated together with structures to allow for the selection of operations, forming the complete ALU.

2.1 Operation Modules

Each operation module accepts the inputs, and supplies the outputs shown in table 3. All outputs of the modules are routed through tristate buffers - the outputs of an operation are floating unless its enable pin is high.

There are two primary types of operation modules utilized in this ALU, combinational logic operations, and sequential logic operations. For combinational logic operations, the CLK and RST_N inputs are not needed, and DONE is always high when the module is enabled. A diagram of such an operation module is shown in figure 3. Sequential logic operations utilize all inputs to perform operations that require more than one clock cycle. The only such operation in this ALU is multiplication. A diagram of this type of module is shown in 4.

The following sections will include a brief description of the implementation of each ALU operation, and a listing of its SystemVerilog code.

2.1.1 A Pass

A Passthrough passes the 8-bit operand A to the output Y without modification. In the case enable is low, all outputs float to Z.

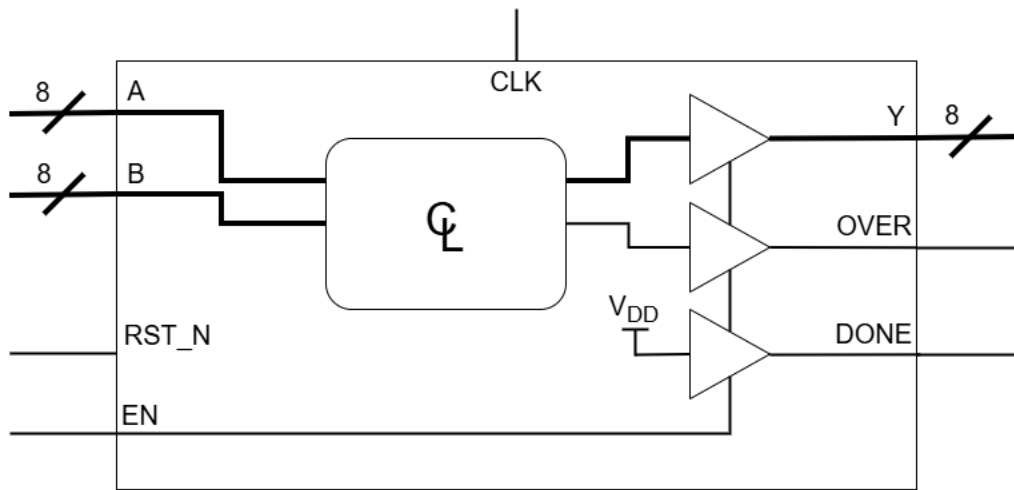


Figure 3: Schematic of a Combinational Operation Module

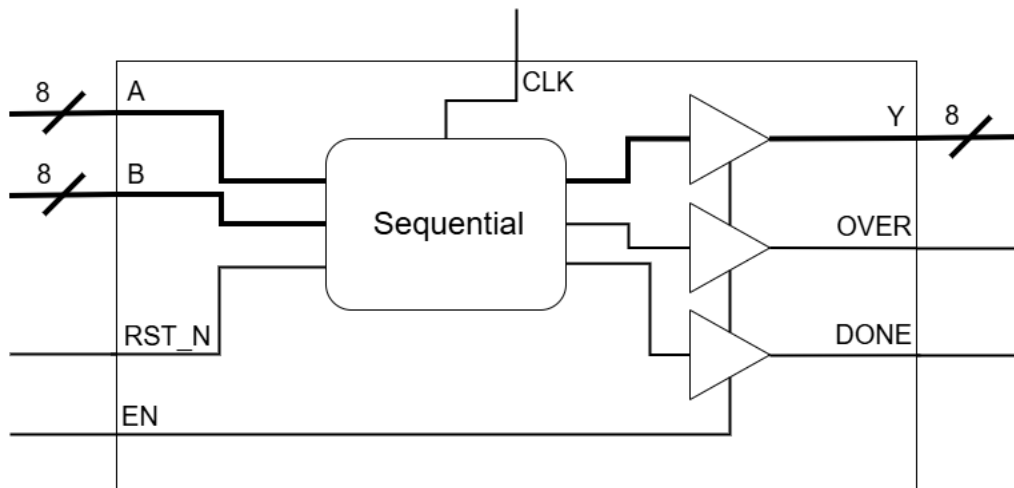


Figure 4: Schematic of a Sequential Operation Module

Listing: a_pass.sv

```

1  /*
2  Project: 8-Bit ALU
3  Author: John O'Connor
4  Sources: N/A
5
6  Module for A passthrough
7  */
8
9  module A_pass(
10     input logic [7:0] A,
11     input logic enable,
12     output logic [7:0] Y,
13     output logic done,
14     output logic overflow
15
16 );

```

Bus / Pin	Width (Bits)	Function
Input		
A	8	First 8-Bit operand - input to operation.
B	8	Second 8-Bit operand - input to operation.
RST_N	1	Active low asynchronous reset - resets the module to initial ready state.
EN	1	Enable - enables the module. The module will only output data when this pin is high
Output		
Y	8	8 Bit result - output of the operation
OVER	1	Overflow flag - high value generally indicates result cannot be represented in bits of Y. Exact meaning can vary between operations.
DONE	1	Is low while operation is in progress or is ready, high when operation is completed.

Table 3: Operation Module IO Descriptions

```

17
18     assign Y = enable ? A : 8'bz; //Assigns Y when enable is high
19     assign done = enable ? 1'b1 : 1'bz; // Done is high when enable is high
20     assign overflow = enable ? 1'b0 : 1'bz; // No overflow occurs for passthrough
21
22
23
24 endmodule

```

2.1.2 B Pass

B Passthrough passes the 8-bit operand B to the output Y without modification. In the case enable is low, all outputs float to Z.

Listing: b_pass.sv

```

1  /*
2  Project: 8-Bit ALU
3  Author: John O'Connor
4  Sources: N/A
5
6  Module for B passthrough
7  */
8
9  module B_pass(
10     input logic [7:0] B,
11     input logic enable,

```

```

12     output logic [7:0] Y,
13     output logic done,
14     output logic overflow
15 );
16
17     assign Y = enable ? B : 8'bz; //Assigns Y when enable is high
18     assign done = enable ? 1'b1 : 1'bz; // Done is high when enable is high
19     assign overflow = enable ? 1'b0 : 1'bz; // No overflow occurs for passthrough
20
21
22 endmodule

```

2.1.3 Bitwise AND

Bitwise AND performs the logical AND operation on each corresponding bit of the operands A and B, setting the result of this to Y. A bit in Y is 1 if and only if the corresponding bits in A and B are both 1. When enable is low, all outputs float.

Listing: bw_and.sv

```

1  /*
2  Project: 8-Bit ALU
3  Author: Max Fabian
4  Sources: N/A
5
6  Module for running the Bitwise AND in the ALU
7  */
8  module Bitwise_AND(
9      input logic [7:0] A, B, //8-bit Operand inputs
10     input logic enable,      //Enable - sets output of operator to floating
11     output logic [7:0] Y,    //8-bit Output
12     output logic done,       //Done Flag
13     output logic overflow    //Overflow Flag
14 );
15
16     //Assigns Y to the output of Bitwise AND when enable is high
17     assign Y = enable ? (A & B) : 8'bz;
18
19     //Simple Functions like this one have simple done and overflow flags
20     assign done      = enable ? 1'b1      : 1'bz;
21     assign overflow  = enable ? 1'b0      : 1'bz;
22
23 endmodule

```

2.1.4 Bitwise OR

Bitwise OR performs the logical OR operation on each corresponding bit of the operands A and B, setting the result of this to Y. A bit in Y is 1 if either or both corresponding bits in A and B are 1. When enable is low, all outputs float.

Listing: bw_or.sv

```
1  /*
2  Project: 8-Bit ALU
3  Author: Max Fabian
4  Sources: N/A
5
6  Module for running the Bitwise OR in the ALU
7  */
8  module Bitwise_OR (
9      input logic [7:0] A, B, //8-bit Operand inputs
10     input logic enable,      //Enable - sets output of operator to floating
11     output logic [7:0] Y,     //8-bit Output
12     output logic done,        //Done Flag
13     output logic overflow     //Overflow Flag
14 );
15
16     //Assigns Y to the output of Bitwise OR when enable is high
17     assign Y = enable ? (A | B) : 8'bz;
18
19     //Simple Functions like this one have simple done and overflow flags
20     assign done = enable ? 1'b1:1'bz;
21     assign overflow = enable ? 1'b0:1'bz;
22
23 endmodule
```

2.1.5 Bitwise XOR

Bitwise XOR performs the logical XOR operation on each corresponding bit of the operands A and B, setting the result of this to Y. A bit in Y is 1 if either the of the corresponding bits in A or B is 1 but both. When enable is low, all outputs float.

Listing: bw_xor.sv

```
1  /*
2  Project: 8-Bit ALU
3  Author: John O'Connor
4  Sources: N/A
5
6  Module for Bitwise XOR.
7  */
8
9  module Bitwise_XOR (
10     input logic [7:0] A, B,
11     input logic enable,
12     output logic [7:0] Y,
13     output logic done,
14     output logic overflow
15 );
16
17
18     assign Y = enable ? (A ^ B) : 8'bz; // Assigns Y when enable is high, otherwise Y is
    ↪ floating
```



```

19     assign done = enable ? 1'b1 : 1'bz; // Done is high when enable is high
20     assign overflow = enable ? 1'b0 : 1'bz; // No overflow occurs with bitwise XOR
21
22 endmodule

```

2.1.6 Addition

Addition performs unsigned addition on two 8-bit operands A and B, setting the result of this to Y. In the case where the result of $A + B$ exceeds 255, overflow will be set to 1, representing the most significant bit. When enable is low, all outputs float.

Listing: addition.sv

```

1  /*
2  Project: 8-Bit ALU
3  Author: Max Fabian
4  Sources: N/A
5
6  Module for running the adder in the ALU
7  */
8  module Addition(
9      input logic [7:0] A, B, //8-bit operand inputs
10     input logic enable,      //Enable - sets output of operator to floating
11     output logic [7:0] Y,    //8-bit output
12     output logic overflow,   //Overflow Flag
13     output logic done        //Done Flag
14 );
15     //Assigns overflow as 9th bit to Y and takes adder when enable is high
16     assign {overflow, Y} = enable ? (A + B) : 9'bz;
17
18     //Done flag on enable high
19     assign done = enable ? 1'b1:1'bz;
20
21 endmodule

```

2.1.7 Subtraction

Subtraction performs unsigned subtraction on two 8-bit operands A and B, setting the result of this to Y. This implementation uses the two's complement of B and computes subtraction by adding A to the two's complement of B. When enable is low, all outputs float.

Listing: subtraction.sv

```

1  /*
2  Project: 8-Bit ALU
3  Author: Max Fabian
4  Sources: N/A
5
6  Module for running the Subtractor in the ALU

```

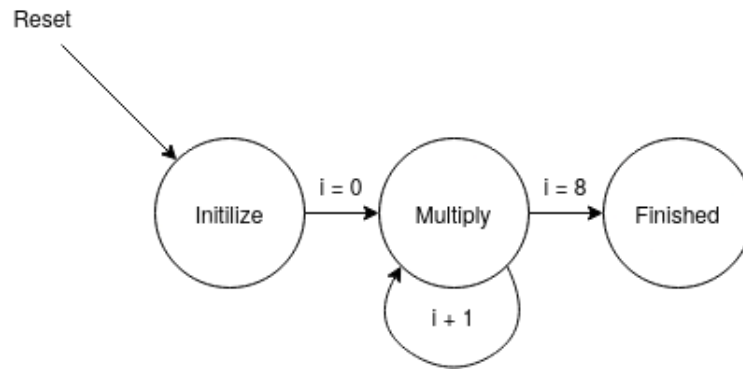



Figure 6: State Machine Diagram for the Multiplication Module

Listing: mult.sv

```

1  /*
2  Project: 8-Bit ALU
3  Author: Max Fabian
4  Sources:
5      *https://en.wikipedia.org/wiki/Binary_multiplier#Single-cycle_multiplier
6
7      ↪ *https://stackoverflow.com/questions/63655634/8-bit-sequential-multiplier-using-add-and-shift
8  Sequential Multiplier, functions similarly to doing long multiplication by hand
9  */
10 module Mult_8bit (
11     input logic [7:0] A, B, //8-bit Operand inputs
12     input logic clock,      //clock to allow for sequential logic
13     input logic reset_n,    //Resets internal registers to base state
14     input logic enable,     //Enable - sets output of operator to floating
15     output logic [7:0] Y,   // 8-bit Output
16     output logic done,      //Done Flag
17     output logic overflow   //Overflow Flag
18 );
19
20 //Registers for helping track the current sequence in the multiplication
21 logic [15:0] result;
22 logic [15:0] multiplicand;
23 logic [3:0] i;
24
25 //Wire for the highest 8 bits of result, to help deconstruct overflow
26 logic [7:0] cout;
27
28 //Needs to trigger only when enable is on or when resetting
29 always @(posedge (clock && enable) or negedge reset_n or negedge(enable)) begin
30     //Resets all the registers and the done flag to initial states
31     if(~reset_n) begin
32         result      <= 0;
33         multiplicand <= A;
34         i           <= 0;
35
36         //Functions as equivalent to enable to them to function reasonably
37         // asynchronous
  
```

```

38         Y           <= 8'bz;
39         done        <= 1'bz;
40         overflow    <= 1'bz;
41     end
42     //Sets the outputs to floating when the enable is off
43     else if(~enable)begin
44         Y           <= 8'bz;
45         done        <= 1'bz;
46         overflow    <= 1'bz;
47     end else begin
48         //For loop created using a counter and comparitor
49         //Functionally same as:
50         // for(int i = 0; i < 8; i++)
51         // counts from 0 to 7 inclusively
52         if(i < 8) begin
53             //Performed on every iteration so the current
54             // sequence in multiplication is correct
55             multiplicand <= multiplicand << 1; //Logical Left Shift
56                                                     //I.E. multiplied by 2
57
58             //Reads each bit of B and increases the result by the
59             // current left shifted multiplicand
60             if(B[i] == 1) begin
61                 result <= result + multiplicand;
62             end
63             //Iterand counting by one to complete for loop functionality
64             i <= i + 1;
65
66             //Once the loop has finished the result can be thrown to
67             // the outputs, and the done flag can be raised so the ALU
68             // knows the operation has concluded.
69         end else begin
70             {cout, Y} = result;
71             //Since the output of a multiplier is 2x number of
72             // input bits and our output is staying at 8 bits
73             // the overflow flag needs to be raised whenever
74             // the result exceeds 8 bits
75
76             if(cout > 0) begin
77                 overflow <= 1;
78             end else begin
79                 overflow <= 0;
80             end
81
82             //Raising the done flag should always be the last
83             // thing it does
84             done <= 1'b1;
85         end
86     end
87 end
88
89 endmodule
90
91

```

2.2 ALU Utility Modules

Three SystemVerilog modules were developed to be used within ALU operations or the ALU itself. These include a variable width asynchronously resettable register with enable, a 3 bit multiplexer with enable, and a variable width counter. The Implementation details and SystemVerilog listings for these modules are provided in the subsequent sections.

2.2.1 Opcode Decoder

The decoder takes a 3-bit opcode input and converts it to an 8-bit, one-hot signal used to enable a given operation module. When enable is low, all outputs are set to 0.

Listing: multiplexer_3_en.sv

```
1  /*
2  Project: 8-Bit ALU
3  Author: Nolan Kessler
4
5  Implements a three bit multiplexer with enable
6  */
7
8  module Decoder (
9      input logic [2 : 0] in,
10     input logic enable, //Output of multiplexer is 0 unless enable is high.
11     output logic [7 : 0] out
12 );
13     always_comb begin
14         if (enable) begin
15             case (in)
16                 3'b000: out = 8'b0000_0001;
17                 3'b001: out = 8'b0000_0010;
18                 3'b010: out = 8'b0000_0100;
19                 3'b011: out = 8'b0000_1000;
20                 3'b100: out = 8'b0001_0000;
21                 3'b101: out = 8'b0010_0000;
22                 3'b110: out = 8'b0100_0000;
23                 3'b111: out = 8'b1000_0000;
24             endcase
25         end
26         else begin
27             out = 4'b0;
28         end
29     end
30 endmodule
```

2.2.2 Variable Width Register

The variable width register is a configurable register, with a default width of 1, used to store the value of the in signal. If enable is high, the input "in" is copied to the output on the rising clock edge. Then, when enable is low, the register holds the value of out. If reset goes low, the output is immediately set to 0.

Listing: decoder.sv

```
1  /*
2  Project: 8-Bit ALU
3  Author: Nolan Kessler
4  Sources: Based on my register implementation from Lab 5
5
6  Implements a parameterized width register with enable and active low reset.
7  */
8
9  module Register_en_rstn #(parameter WIDTH = 1) (
10     input logic clk,
11     input logic enable, //Enable - copy in to out on rising edge of clk only when high
12     input logic rst_n, //Active low reset - set out to zero on falling edge
13     input logic [WIDTH - 1 : 0] in,
14     output logic [WIDTH - 1 : 0] out
15 );
16
17     always_ff @(posedge clk, negedge rst_n) begin
18
19         if (rst_n == 0) begin
20             out <= 0;
21         end
22
23         else begin
24             if (enable != 0) begin
25                 out <= in;
26             end
27         end
28
29     end
30
31 endmodule
```

2.2.3 Counter

The counter is an N width counting module with a default width of 4 bits. On the rising clock edge, the counter increments itself by the value of addBy. If reset is high, the counter resets the count to 0.

Listing: counter.sv

```
1  module CounterNbit #(parameter N = 3)(
2     input logic clock,
3     input logic reset_n, enable_n,
4     input logic [N:0] addBy,
5     output logic [N:0] count
6 );
7
8     logic [N:0] count_next;
9
10     always_comb begin //@(posedge clock || reset_n) begin
```

```

11     if (reset_n == 1'b0) begin
12         count_next <= count + addBy;
13     end else begin
14         count_next <= 0;
15     end
16 end
17
18 Register_en_rstn #(.WIDTH(N+1)) RegNBit (
19     .clk(clock),
20     .rst_n(reset_n), .enable(enable_n),
21     .in(count_next),
22     .out(count)
23 );
24
25 endmodule
26

```

2.3 ALU Implementation

A schematic diagram of the full ALU is shown in figure 7. Four registers are used to ensure that the inputs to the ALU remain consistent throughout the entire operation. R_a , R_b , and R_{op} will store the inputs to the ALU on the first rising clock edge that occurs while EN is high. The output of the locking register R_{lk} becomes high on the next clock falling edge, disabling further updates to A, B, and OP, and enabling the decoder, which will select the requested function. At this point the value of EN can be changed without impacting the ALU operation.

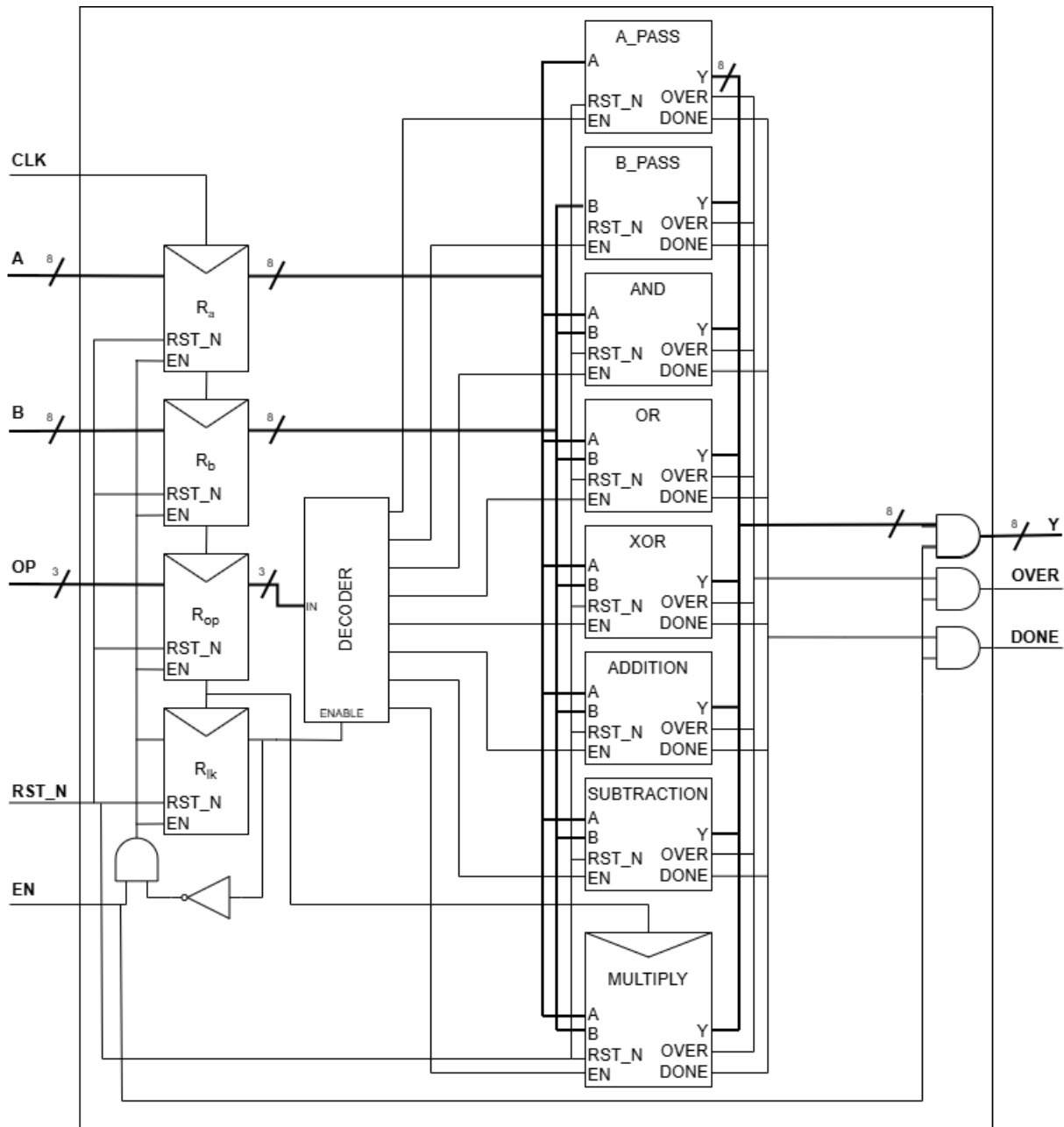


Figure 7: Schematic of Full ALU

3 Project Reflection

Implementing the project overall went well. Implementing each individual combinational module and getting them working in the larger ALU module was quick and relatively simple. However, we ran into some complications when attempting to implement a more complicated sequential module. This was the multiplication module. Since it required multiple clock cycles to perform, the module needed to have proper timing and a comparator to track the current clock cycle the system is on and run the system accordingly. We ran into a few issues with the done and overflow registers being out of sync causing the module to fail to properly output the overflow flag. These smaller difficulties were more time-consuming than difficult as they were small errors that required

looking over the HDL script and RTL Viewer to identify where the timing issues would occur.

4 Design Verification

Design verification is done using SystemVerilog test benches. Each operation module has its own test bench to test expected values compared to simulated. The ALU itself has its own test bench which ensures the ALU design and utility modules function as expected.

4.1 Operation Test Benches

Each operation module has a test bench. All test benches with the exception of the multiplication test bench test all possible combinations of operands as well as testing the case where the enable signal is low.

4.1.1 A Pass

Listing: a_pass.tb.sv

```
1  /*
2  Project: 8-Bit ALU
3  Author: John O'Connor
4  Sources: N/A
5
6  Testbench file for the A passthrough module (A_pass.sv).
7  */
8
9  module A_pass_tb;
10     // Inputs
11     logic [7:0] A;
12     logic enable;
13     // Outputs
14     logic [7:0] Y;
15     logic done;
16     logic overflow;
17
18     A_pass dut(
19         .A(A),
20         .enable(enable),
21         .Y(Y),
22         .done(done),
23         .overflow(overflow)
24     );
25
26     initial begin
27
28         // Test case where enable is low
29         enable = 0;
30
31         A = 8'b0;
32         #10;
33         // Y, done, and overflow should all be floating
```

```

34         if (Y !== 8'bz || done !== 1'bz || overflow !== 1'bz) begin
35             $display("Test Failed enable low case");
36             $stop;
37         end
38
39         // Test cases where enable is high
40         enable = 1;
41
42         // Test all possible values for A
43         // Expected: Y = A for every test
44         for (int i=0; i<256; i++) begin
45             A = i;
46             #10;
47             if (Y !== A || done !== 1'b1 || overflow !== 1'b0) begin
48                 $display("Test Failed for A = %0d", i);
49                 $stop;
50             end
51         end
52
53         $display("All Tests Passed");
54
55     end
56
57 endmodule

```

4.1.2 B Pass

Listing: b_pass.tb.sv

```

1  /*
2  Project: 8-Bit ALU
3  Author: John O'Connor
4  Sources: N/A
5
6  Testbench file for the B passthrough module (b_pass.sv).
7  */
8
9  module B_pass_tb;
10     // Inputs
11     logic [7:0] B;
12     logic enable;
13     // Outputs
14     logic [7:0] Y;
15     logic done;
16     logic overflow;
17
18     B_pass dut(
19         .B(B),
20         .enable(enable),
21         .Y(Y),
22         .done(done),
23         .overflow(overflow)
24     );

```

```

25
26     initial begin
27
28         // Test case where enable is low
29         enable = 0;
30
31         B = 8'b0;
32         #10;
33         // Y, done, and overflow should all be floating
34         if (Y !== 8'bz || done !== 1'bz || overflow !== 1'bz) begin
35             $display("Test Failed enable low case");
36             $stop;
37         end
38
39         // Test cases where enable is high
40         enable = 1;
41
42         // Test every possible value for B
43         // Expected: Y = B for each value
44         for (int i=0; i<256; i++) begin
45             B = i;
46             #10;
47             // Check Y is correct, done is high, overflow is low
48             if (Y !== B || done !== 1'b1 || overflow !== 1'b0) begin
49                 $display("Test Failed for B = %0d", i);
50                 $stop;
51             end
52         end
53
54         $display("All Tests Passed");
55
56     end
57
58 endmodule

```

4.1.3 Bitwise AND

Listing: and.tb.sv

```

1  /*
2  Project: 8-Bit ALU
3  Author: John O'Connor
4  Sources: N/A
5
6  Testbench file for the And module (bw_and.sv).
7  */
8
9  module And_tb;
10     // Inputs
11     logic [7:0] A, B;
12     logic enable;
13     // Outputs
14     logic [7:0] Y;

```

```

15     logic done;
16     logic overflow;
17
18     Bitwise_AND dut(
19         .A(A),
20         .B(B),
21         .enable(enable),
22         .Y(Y),
23         .done(done),
24         .overflow(overflow)
25     );
26
27     initial begin
28
29         // Test case where enable is low
30         enable = 0;
31         A = 8'b0;
32         B = 8'b1;
33         #10;
34         // Y, done, and overflow should all be floating
35         if (Y !== 8'bz || done !== 1'bz || overflow !== 1'bz) begin
36             $display("Test Failed enable low case");
37             $stop;
38         end
39
40         // Test cases where enable is high
41         enable = 1;
42
43         // Test all possible values for A and B
44         for(int i=0; i<256; i++) begin
45             for (int j=0; j<256; j++) begin
46                 A = i;
47                 B = j;
48                 #10;
49                 // Check Y is correct, done is high, and overflow is low
50                 if (Y!==(A&B) || done !== 1'b1 || overflow !== 1'b0) begin
51                     $display("Test Failed");
52                     $stop;
53                 end
54             end
55         end
56
57         $display("All Tests Passed");
58     end
59
60 endmodule

```

4.1.4 Bitwise OR

Listing: bw_or.tb.sv

```

1  /*
2  Project: 8-Bit ALU

```

```

3 Author: Max Fabian
4 Sources: N/A
5
6 Testbench file for the OR module (bw_or.sv).
7 */
8
9 module Or_tb();
10
11     //Module Inputs
12     logic [7:0] A_input, B_input;
13     logic enable;
14
15     //Module Outputs
16     logic [7:0] Y_test;
17     logic done, overflow;
18
19     //Module Call
20     Bitwise_OR dut(
21         .A(A_input), .B(B_input),
22         .enable(enable),
23         .Y(Y_test),
24         .overflow(overflow), .done(done)
25     );
26
27     initial begin
28
29         // Test case where enable is low
30         enable = 0;
31         A_input = 8'b0;
32         B_input = 8'b1;
33         #10;
34         // Y, done, and overflow should all be floating
35         if (Y_test !== 8'bz || done !== 1'bz || overflow !== 1'bz) begin
36             $display("Test Failed enable low case");
37             $stop;
38         end
39
40         // Test cases where enable is high
41         enable = 1;
42
43         // Test all possible values for A and B
44         for(int A = 0; A < 256; A++) begin
45             for(int B = 0; B < 256; B++) begin
46                 A_input = A; B_input = B;
47                 #10;
48                 // Check Y is correct, done is high, and overflow is low
49                 if (Y_test !== (A_input | B_input) || done !== 1'b1 || overflow !== 1'b0)
50                     ↪ begin
51                         $display("Failed at %dA | %dB; Y=%d", A, B, Y_test);
52                         $stop;
53                     end
54             end
55         end
56
57         $display("All Tests Passed");
58     end

```

```
58
59 endmodule
```

4.1.5 Bitwise XOR

Listing: bw_xor.tb.sv

```
1  /*
2  Project: 8-Bit ALU
3  Author: John O'Connor
4  Sources: N/A
5
6  Testbench file for the Bitwise XOR (bw_xor.sv).
7  */
8
9  module Xor_tb;
10     // Inputs
11     logic [7:0] A, B;
12     logic enable;
13     // Outputs
14     logic [7:0] Y;
15     logic done;
16     logic overflow;
17
18     Bitwise_XOR dut(
19         .A(A),
20         .B(B),
21         .enable(enable),
22         .Y(Y),
23         .done(done),
24         .overflow(overflow)
25     );
26
27     initial begin
28
29         // Test case where enable is low
30         enable = 0;
31
32         A = 8'b0;
33         B = 8'b1;
34         #10;
35         // Y, done, and overflow should all be floating
36         if (Y !== 8'bz || done !== 1'bz || overflow !== 1'bz) begin
37             $display("Test Failed enable low case");
38             $stop;
39         end
40
41         // Test cases where enable is high
42         enable = 1;
43
44         // Test every possible combination of A and B
45         for(int i=0; i<256; i++) begin
46             for (int j=0; j<256; j++) begin
```

```

47         A = i;
48         B = j;
49         #10;
50         // Check Y is correct, done is high, and overflow is low
51         if (Y!= (A^B) || done !== 1'b1 || overflow !== 1'b0) begin
52             $display("Test Failed");
53             $stop;
54         end
55     end
56 end
57
58     $display("All Tests Passed");
59 end
60
61 endmodule

```

4.1.6 Addition

Listing: addition.tb.sv

```

1  module AdditionTestbench();
2
3  //Module Inputs
4  logic [7:0] A_input, B_input;
5  logic enable;
6
7  //Module Outputs
8  logic [7:0] Y_test;
9  logic done, overflow;
10
11 //Expected Outputs
12 logic [7:0] Y_exp;
13 logic overflow_exp;
14
15 Addition dut(
16     .A(A_input), .B(B_input),
17     .enable(enable),
18     .Y(Y_test),
19     .overflow(overflow), .done(done)
20 );
21
22 initial begin
23
24     // Test case where enable is low
25     enable = 0;
26     A_input = 8'b0;
27     B_input = 8'b1;
28     #10;
29     // Y, done, and overflow should all be floating
30     if (Y_test !== 8'bz || done !== 1'bz || overflow !== 1'bz) begin
31         $display("Test Failed enable low case");
32         $stop;
33     end

```

```

34
35 // Test cases where enable is high
36 enable = 1;
37
38 // Test all possible values for A and B
39 for(int A = 0; A < 256; A++) begin
40     for(int B = 0; B < 256; B++) begin
41         A_input = A; B_input = B;
42         {overflow_exp, Y_exp} = A + B;
43         #10;
44         // Check Y is correct, done is high, and overflow is correct
45         if (Y_test !== Y_exp || done !== 1'b1 || overflow !== overflow_exp) begin
46             $display("Failed at %dA + %dB; Y=%d and Y_exp=%d and overflow=%d and
47                 ↪ overflow_exp=%d",
48                 A, B, Y_test, Y_exp, overflow, overflow_exp);
49             $stop;
50         end
51     end
52 end
53 $display("All Tests Passed");
54 end
55
56 endmodule

```

4.1.7 Subtraction

Listing: subtraction.tb.sv

```

1 module SubtractionTestbench();
2
3 //Module Inputs
4 logic [7:0] A_input, B_input;
5 logic enable;
6
7 //Module Outputs
8 logic [7:0] Y_test;
9 logic done, overflow;
10
11 //Expected Outputs
12 logic [7:0] Y_exp;
13
14 //Module Call
15 Subtraction dut(
16     .A(A_input), .B(B_input),
17     .enable(enable),
18     .Y(Y_test),
19     .overflow(overflow), .done(done)
20 );
21
22 initial begin
23
24     // Test case where enable is low

```



```

25     enable = 0;
26     A_input = 8'b0;
27     B_input = 8'b1;
28     #10;
29     // Y, done, and overflow should all be floating
30     if (Y_test !== 8'bz || done !== 1'bz || overflow !== 1'bz) begin
31         $display("Test Failed enable low case");
32         $stop;
33     end
34
35     // Test cases where enable is high
36     enable = 1;
37
38     // Test all possible values for A and B
39     for(int A = 0; A < 256; A++) begin
40         for(int B = 0; B < 256; B++) begin
41             A_input = A; B_input = B;
42             Y_exp = A - B;
43             #10;
44             // Check Y is correct, done is high, overflow is always 0 for subtraction
45             if (Y_test !== Y_exp || done !== 1'b1 || overflow !== 1'b0) begin
46                 $display("Failed at %dA - %dB; Y=%d and Y_exp=%d",
47                     A, B, Y_test, Y_exp);
48                 $stop;
49             end
50         end
51     end
52
53     $display("All Tests Passed");
54 end
55
56 endmodule

```

4.1.8 Multiplication

Listing: mult.tb.sv

```

1  module Mult_8bitTestbench();
2
3  //Module Inputs
4  logic [7:0] A_input, B_input;
5  logic enable, clock, reset_n;
6
7  //Module Outputs
8  logic [7:0] Y_test;
9  logic cout_test, Done_test;
10
11 //Expected Outputs
12 logic [7:0] Y_exp;
13 logic cout_exp, Done_exp;
14
15 //Additional Logic to Help account for overflow
16 logic [7:0] overflow_catch;

```

```

17
18 //Module Call
19 Mult_8bit dut(
20     .A(A_input), .B(B_input),
21     .enable(enable), .clock(clock), .reset_n(reset_n),
22     .Y(Y_test),
23     .overflow(cout_test), .done(Done_test)
24 );
25
26
27 task reset();
28 /*
29  * Reset Task:
30  *   Runs the reset pin at active low
31  *   Should reset all registers to 0 and set done flag to 0
32  *   And sets outputs to floating
33  */
34     reset_n = 1'b0;
35     #10;
36     reset_n = 1'b1;
37 endtask
38
39 task multiply();
40 /*
41  * Multiply Task:
42  *   Since the multiplier Module runs on a number of clock cycles
43  *   The test case needs to be predictable for proper comparison
44  *
45  *   Therefore the multiply function runs the clock until the Done flag is
46  *   raised. Allowing the multiplier enough time to complete the
47  *   multiplication.
48  */
49     while (Done_test !== 1'b1) begin
50         clock = 1'b1;
51         #1;
52         clock = 1'b0;
53         #1;
54     end
55 endtask
56
57 task validate();
58 /*
59  * Validate Task:
60  *   Iterates through a limited set of values for A and B
61  *   and compares the calculated values of multiplication
62  *   to the implemented addition module
63  *
64  *   Since the output is limited to 8-bits the multiplier
65  *   technically only works for two 4-bit inputs,
66  *   and the each test takes 9 clock cycles
67  *   so the test is limited to a 6-bit maximum
68  */
69
70     int A2; //Declare variables for use in later test case
71     int B2;
72

```

```

73 for(int A = 0; A < 64; A++) begin
74     //If enable is off we should see a floating output
75     if(enable == 0) begin
76         //Running multiply here causes the module to run forever
77         // Which proves that it works correctly
78         // However this is also why it is commented out.
79
80         //multiply();
81         $display("Function not yet enabled %dY_test %dcout_test",
82             Y_test, cout_test);
83         break;
84     end
85     for(int B = 0; B < 64; B++) begin
86         //Assigns the module bus inputs to current values
87         A_input = A; B_input = B;
88         //Calculates expected values and outputs them to expected
89         {overflow_catch, Y_exp} = A * B;
90         //Overflow Catch exists because of the output size limitation
91         // always overflows when the output is bigger than the output
92         // instead of only when the 9th bit is 1 like for other TBs
93         if(overflow_catch > 0) begin cout_exp = 1; end
94
95         multiply(); //Run the multiply task to give enough clock cycles
96         #5;         //Pause to let module process
97
98         //Checks all expected to tested from module, shows where it fails
99         if(cout_exp != cout_test && Y_exp != Y_test && Done_test != Done_exp) begin
100             $display("Failed at %dA * %dB; Y=%d and Y_exp=%d and cout=%d and cout_exp=%d",
101                 A, B, Y_test, Y_exp, cout_test, cout_exp);
102             //Does not stop at fail to identify the pattern of failure
103             // from test results
104         end
105         reset();
106     end
107 end
108 if(enable == 1) begin
109     //This is a special case test for when the value should be
110     // massively large but will appear much smaller due to the limitation
111     // of the output.
112     //
113     //Same as the looped cases above
114     A2 = 255;
115     B2 = 128;
116     A_input = A2;
117     B_input = B2;
118     {cout_exp, Y_exp} = A2 * B2;
119
120     multiply();
121     #5;
122
123     if(cout_exp != cout_test && Y_exp != Y_test && Done_test != Done_exp) begin
124         $display("Failed at %dA * %dB; Y=%d and Y_exp=%d and cout=%d and cout_exp=%d",
125             A2, B2, Y_test, Y_exp, cout_test, cout_exp);
126         //$stop;
127     end
128     reset();

```

```

129     end
130 endtask
131
132 initial begin
133     //Initialize Variables
134     enable = 1'b0; //First test enable at 0
135     A_input = 0;
136     B_input = 0;
137     Done_exp= 1;    //Expected Done flag will always be 1
138
139     reset();        //Reset to begin, otherwise last ran test will be output
140     validate();     //Should display the not yet enabled message
141
142     enable = 1'b1;  //Now test enable at 1
143
144     reset();        //Reset for same reason as before
145     validate();     //Should pass with no error
146
147     #100;           //Take a breather
148     $stop;
149 end
150
151 endmodule
152
153
154
155

```

4.2 ALU Test Bench

The ALU test bench tests the ALU design as well as the functionality of the utility modules. It iterates through each operation by testing one opcode at a time, resetting between tests.

Listing: alu.tb.sv

```

1 module ALU_tb;
2     //inputs
3     logic    en;
4     logic    clk;
5     logic    rst_n;
6     logic [7 : 0] a, b;
7     logic [2 : 0] op;
8     //outputs
9     logic [7:0] y;
10    logic    overflow;
11    logic    done;
12
13    alu dut (
14        .en(en),
15        .clk(clk),
16        .rst_n(rst_n),
17        .a(a),

```

```

18     .b(b),
19     .op(op),
20     .y(y),
21     .overflow(overflow),
22     .done(done)
23 );
24
25 always #5 clk = ~clk;
26
27 // task to reset between operations
28 task reset();
29     en = 0;
30     rst_n = 0;
31     #10;
32     rst_n = 1;
33 endtask
34
35 // executing task
36 task execute();
37     @(negedge clk);
38     en = 1;
39     #5;
40     @(posedge done);
41     #5;
42 endtask
43
44 initial begin
45     clk = 0;
46     a = 0;
47     b = 0;
48     op = 0;
49     en = 0;
50     reset();
51
52     // a_pass (0x0)
53     reset();
54     a = 8'hAA; b = 8'h55; op = 4'h0;
55     execute();
56     if (y !== 8'hAA || done !== 1'b1 || overflow !== 1'b0) begin
57         $display("FAILED: A_pass");
58         $stop;
59     end
60
61     // B_pass (0x1)
62     reset();
63     a = 8'hAA; b = 8'h55; op = 4'h1;
64     execute();
65     if (y !== 8'h55 || done !== 1'b1 || overflow !== 1'b0) begin
66         $display("FAILED: B_pass");
67         $stop;
68     end
69
70     // Bitwise AND (0x2)
71     reset();
72     a = 8'hAA; b = 8'h55; op = 4'h2;
73     execute();

```

```

74     if (y !== 8'h00 || done !== 1'b1 || overflow !== 1'b0) begin
75         $display("FAILED: Bitwise AND");
76         $stop;
77     end
78
79     // Bitwise OR (0x3)
80     reset();
81     a = 8'hAA; b = 8'h55; op = 4'h3;
82     execute();
83     if (y !== 8'hFF || done !== 1'b1 || overflow !== 1'b0) begin
84         $display("FAILED: Bitwise OR");
85         $stop;
86     end
87
88     // Bitwise XOR (0x4)
89     reset();
90     a = 8'hAA; b = 8'h55; op = 4'h4;
91     execute();
92     if (y !== 8'hFF || done !== 1'b1 || overflow !== 1'b0) begin
93         $display("FAILED: Bitwise XOR");
94         $stop;
95     end
96
97     // Addition (0x5)
98     reset();
99     a = 8'h01; b = 8'h02; op = 4'h5;
100    execute();
101    if (y !== 8'h03 || done !== 1'b1 || overflow !== 1'b0) begin
102        $display("FAILED: Addition");
103        $stop;
104    end
105
106    // Addition with overflow (0x5)
107    reset();
108    a = 8'hFF; b = 8'h01; op = 4'h5;
109    execute();
110    if (y !== 8'h00 || done !== 1'b1 || overflow !== 1'b1) begin
111        $display("FAILED: Addition overflow");
112        $stop;
113    end
114
115    // Subtraction (0x6)
116    reset();
117    a = 8'h05; b = 8'h03; op = 4'h6;
118    execute();
119    if (y !== 8'h02 || done !== 1'b1 || overflow !== 1'b0) begin
120        $display("FAILED: Subtraction");
121        $stop;
122    end
123
124    // Multiplication (0x7)
125    reset();
126    a = 8'h05; b = 8'h03; op = 4'h7;
127    execute();
128    if (y !== 8'h0F || done !== 1'b1 || overflow !== 1'b0) begin
129        $display("FAILED: Multiplication");

```

```

130         $stop;
131     end
132
133     $display("All Tests Passed!");
134     $stop;
135 end
136 endmodule

```

4.3 Simulation Testing

Simulation can be done using ModelSim. To accompany this, we can run a do file which runs all test benches.

Listing: ALUSim.do

```

1  # Project: 8-Bit ALU
2  # Author: John O'Connor
3  # Sources: None
4
5  # Do file for running all independent testbenches
6
7  #-nostdout
8  # Run module testbenches
9  vsim -quiet A_pass_tb -do "run -all"
10 vsim -quiet B_pass_tb -do "run -all"
11 vsim -quiet And_tb -do "run -all"
12 vsim -quiet Or_tb -do "run -all"
13 vsim -quiet Xor_tb -do "run -all"
14 vsim -quiet AdditionTestbench -do "run -all"
15 vsim -quiet SubtractionTestbench -do "run -all"
16 vsim -quiet Mult_8bitTestbench -do "run -all"
17
18 # ALU testbench
19 vsim ALU_tb
20 add wave *
21 run -all

```

ALU Testbench Waveform

We can analyze the ALU test bench's waveform using the waveform

5 Hardware Validation

6 Reflection on Design Process